

Day 5 — PLC Training

Networking, SCADA & Capstone Line | 8:30 AM – 12:45 PM

Day 5 Syllabus

Duration: 205 min instructional + 15 min break

Prerequisites: Completion of Days 1–4 — ladder logic (Day 1), timers/counters/sequencing (Day 2), analog scaling/comparisons (Day 3), and shift registers/function blocks/PID (Day 4). Today's capstone project deliberately reuses all of it, so any shaky topic from earlier in the week is worth a quick review before class.

Software / Hardware Needed

- OpenPLC Editor (same environment used Days 1–4)
- Projector for live demos
- A second PLC instance or Modbus simulator, to act as a client/server pair for the networking exercise
- Simulated analog load cell input for the capstone sorting line's weight sensing
- A basic HTML/rosbridge-style dashboard template (or the software's built-in SCADA screen) for the tag and alarm exercises

Learning Objectives

By end of Day 5, students should be able to:

1. Explain the client/server relationship in Modbus, and why a server never initiates communication
2. Read a remote analog value into a PLC over a network connection, and correctly scale it once received
3. Explain the distinction between an HMI (single-machine) and a SCADA system (plant-wide, supervisory)
4. Describe what tags, alarms, and trends each add on top of raw networked data
5. Build a tag database and a basic live dashboard displaying named values rather than raw addresses
6. Configure a threshold-based alarm on a live tag
7. Build a complete Sorting & Packaging Line capstone combining shift registers, analog scaling, counters, and a live dashboard with alarms

Topic Breakdown

#	Topic	Time	Type
1	PLC Networking Basics (Modbus)	30 min	Lecture + hands-on
2	HMI vs. SCADA	25 min	Lecture + hands-on
—	Break	15 min	—
3	Capstone Project: Sorting & Packaging Line	150 min	Independent/paired build

Assessment / Checkpoint

By end of day, every student/pair should have a working Sorting & Packaging Line running at least 10 parts of mixed weights end-to-end, with correct live counts and a correctly firing reject alarm shown on the dashboard, demonstrated live to the instructor before leaving — this is also the final checkpoint of the course.

Homework / Next Steps

None required — this is the final day of the course. Students are encouraged to keep their five day-project builds (Motor Start/Stop, Traffic Light, Temperature Monitor, Batch Mixing Station, Sorting & Packaging Line) as a personal reference portfolio covering the full range of ladder logic, timers/counters, analog handling, sequencing, and networking.

Day 5 Learning Content

Networking, SCADA & the Capstone Line — full teaching script

1. PLC Networking Basics (30 min)

Say this to the class:

“Every project this week has lived on one PLC, talking to its own local sensors and lamps. A real plant floor is never just one PLC — it's many PLCs, HMIs, and computers, all needing to see the same data at the same time. Today is about how those separate boxes actually talk to each other, and why that changes how you have to think about your program.”

1.1 — Why a shared network, not just more wires

Wiring every sensor on a large plant directly back to one central PLC would mean enormous cable runs and a single point of failure for the entire facility. Instead, PLCs are distributed around the plant close to the equipment they control, and connected together over a shared digital network — the same physical Ethernet cable can carry data for hundreds of different signals, instead of needing hundreds of separate wires.

1.2 — Modbus: the protocol students will actually touch this week

Modbus is one of the oldest and most common industrial protocols, and it's built on a simple **client/server** relationship (older documentation calls these *master* and *slave* — students will see both terms used interchangeably on real equipment). The client is the device that asks questions; the server is the device that holds the actual data and answers. Crucially, a server never speaks first — it only ever responds when asked.

A Modbus request always asks for data at a specific **address** — for example, “what's the current value at Holding Register 40001?” The server doesn't send back the name of the variable, or any description of what it means — just the raw number stored at that address. All of the meaning (“this is Tank A's temperature”) exists only in the client's own program, not on the wire itself.

Instructor tip: Ask the class: “if a Modbus server never speaks first, what happens if the client asks a question and gets no reply at all?” Answer: it's a genuine hazard, not just a minor delay — a program that blindly uses the last-known value without checking for a timeout can keep acting on stale data indefinitely after a real network fault, exactly the kind of silent failure the 4-20mA discussion on Day 3 was designed to avoid at the sensor level.

Worked example — reading a remote tank level over Modbus

```
Client (main PLC)  --- "read Holding Register 40010" --->  Server (remote I/O module)
Client (main PLC)  <--- "value = 673"                      ---  Server (remote I/O module)
```

The main PLC then treats 673 exactly like any other raw analog value from Day 3 - it still needs to be scaled into engineering units before it means anything.

Point this out directly: Modbus doesn't replace anything learned earlier in the week — a value read over the network still arrives as a raw number and still needs the same scaling formula from Day 3 applied to it. Networking only changes *how the number physically arrives*, not what you do with it afterward.

1.3 — Hands-on exercise

1. Configure two PLC instances (or one PLC plus a Modbus simulator) as client and server.
2. Expose a simulated analog value on the server, and read it into the client over Modbus at a known register address.
3. Confirm the value updates on the client side when it's changed on the server side, and measure roughly how much delay exists between the change and it appearing on the client.
4. Extra practice: disconnect the network cable briefly mid-test and observe what the client shows during the outage — does the last value freeze, show an error, or default to zero?

Checkpoint — before moving on

Ask a student: “in a Modbus client/server relationship, which side is allowed to send a message first, without being asked?” Correct answer: only the client — the server only ever responds, it never initiates. This distinction is worth confirming explicitly before moving into HMI/SCADA, since an HMI's connection to a PLC uses this exact same client/server pattern.

2. HMI vs. SCADA (25 min)

Say this to the class:

“You’ve been watching lamps and buttons in a software simulator all week — that’s already a simple form of what’s covered properly now. This section gives the two levels of “screen you look at” in a real plant their proper names, and explains why a plant needs both, not just one.”

2.1 — HMI: local, single-machine

An HMI (Human-Machine Interface) is a screen mounted directly on or near a single machine — the operator standing at that machine uses it to start/stop that machine, see its current status, and clear its faults. An HMI’s job is deliberately narrow: it only needs to know about the one machine it’s attached to.

2.2 — SCADA: plant-wide, supervisory

SCADA (Supervisory Control and Data Acquisition) sits above many HMIs and PLCs at once, typically running on a computer in a control room rather than on the factory floor itself. Where an HMI shows “is this one machine running,” SCADA shows “are all forty machines across the whole plant running, and which one just tripped an alarm.” SCADA reads its data from the same PLCs the HMIs are attached to — usually over the same kind of Modbus (or similar) network just covered — but it collects that data from many PLCs into one unified view.

Instructor tip: A clean framing for students: an HMI answers “what is this machine doing right now?” SCADA answers “what is every machine in the plant doing right now, and what happened an hour ago?” The second question is only answerable with a system that both talks to many PLCs at once and stores history — a single machine’s HMI has no reason to do either.

2.3 — Tags, alarms, and trends — the three things SCADA adds

- **Tags** — a named link between a human-readable label (“Tank_A_Temperature”) and the actual Modbus address it lives at. Without a tag database, every screen would need to reference raw register numbers directly, which is unreadable and fragile if addresses ever change.
- **Alarms** — a rule that watches a tag’s value and flags it when a condition is met (e.g. Tank_A_Temperature exceeding 80°C) — this is the same GT comparison logic from Day 3, just surfaced to an operator’s screen with a timestamp and often a sound, rather than only switching a lamp.
- **Trends** — a running historical log of a tag’s value over time, plotted as a graph, so an operator can see that Tank_A_Temperature has been slowly climbing over the last hour, not just what it reads at this exact instant.

2.4 — Hands-on exercise

1. Build a simple tag database mapping 3-4 friendly names to the Modbus addresses configured in Section 1’s exercise.
2. Build a basic dashboard screen (an HTML/rosbridge-style live view, or the software’s built-in SCADA screen) that displays those tags by name, not by raw address.

3. Configure one alarm — e.g. flag when the simulated tank level tag exceeds a threshold — and confirm it visibly triggers on the dashboard when the condition is met.
4. Extra practice: add a simple trend/graph of one tag updating live as its value changes.

Checkpoint — before moving on

Ask a student: “if a plant has 20 machines, each with its own local HMI, why would it still need a separate SCADA system on top of that?” A good answer touches on: seeing all 20 machines from one place at once, without walking to each one; keeping a history of what happened, which a simple HMI typically doesn't retain; and centralizing alarms so one operator can be notified regardless of which machine is having a problem.

3. Capstone Project: Sorting & Packaging Line (150 min)

Say this to the class:

"This is the last build of the week, and deliberately the biggest one — it pulls together timers and counters from Day 2, analog scaling and comparisons from Day 3, shift registers and function blocks from Day 4, and the tag/dashboard work from the last section, into a single working line. Nothing here is conceptually new; what's new is that nobody is going to tell you which piece to build first."

3.1 — The specification to give students

Write this on the board exactly, and don't pre-solve it for them:

1. Parts arrive on a conveyor and pass an entry sensor, which also reads each part's weight from an analog load cell.
2. A shift register tracks each part's position as it moves through 3 fixed stations on the line (matching the 3 physical inspection/sort points below).
3. At Station 2, the part's scaled weight is compared against two setpoints: parts under 100g are flagged "Light," parts over 500g are flagged "Heavy," and everything in between is "Standard."
4. At Station 3, a diverter arm fires for exactly 1 second (TON) to push "Light" and "Heavy" parts off onto a reject lane, while "Standard" parts continue straight through, uninterrupted.
5. A running count of Standard, Light, and Heavy parts is kept using three separate CTU counters, and all three counts are shown live on a dashboard, alongside an alarm that triggers once Reject count (Light + Heavy combined) exceeds 5 in the current run.

3.2 — The core idea to teach before they start building

This project has one integrating idea worth stating explicitly before anyone touches the software: the shift register isn't just tracking *whether* a part is present at each station, the way it did in the Day 4 conveyor exercise — this time it needs to carry the part's **classification** (Light/Standard/Heavy) along with it, so that by the time a part physically reaches Station 3's diverter, the line still remembers what was decided about it back at Station 2. Ask directly: "if the classification is only calculated once, at Station 2, how does that decision get carried along until the part physically arrives at Station 3, several conveyor steps later?" The answer is that the register needs to shift the classification value itself, not just a single presence bit — a genuine extension of the Day 4 concept, not a repeat of it.

3.3 — Suggested build order

1. Wire the entry sensor, load cell (analog), diverter arm, and all lamps/outputs. Confirm each can be operated and read manually before any sequencing logic exists.
2. Build the weight scaling (Day 3 formula) and the two classification comparisons (Light/Heavy/Standard) in isolation. Sweep a simulated load cell value and confirm the correct classification lamp lights at each threshold, before this is connected to anything moving.
3. Build the shift register to carry both a presence bit and a classification value station-to-station. Test with the line stopped: manually step the advance pulse and confirm classifications ride along correctly with their part, not left behind or reassigned to the wrong slot.

4. Add the diverter's 1-second TON at Station 3, firing only when the carried classification is Light or Heavy, and confirm Standard parts pass through with the diverter never firing.
5. Add the three CTU counters (Standard/Light/Heavy), each pulsed once per part as it's finally sorted at Station 3 — not at Station 2 when it was merely classified.
6. Add the tag database and dashboard from Section 2, displaying all three live counts, and configure the Reject-count-exceeds-5 alarm.
7. Run at least 10 parts of mixed weights through the full line uninterrupted, confirming every part is classified correctly, diverted or passed through correctly, and counted exactly once in the correct counter.

3.4 — Debugging guidance for common failures

- **A part is diverted, but the wrong classification lamp lit for it.** Ask: “when exactly was this part's weight read and classified — at the moment it passed the load cell, or at some other point?” This is almost always the classification being recalculated live at Station 3 using whatever part happens to be on the load cell *right now*, instead of using the value the shift register carried along from Station 2.
- **Counts don't match the number of parts that actually ran.** Ask: “is the counter being pulsed once per part sorted, or once per scan while some condition happens to stay true?” Exactly the same rising-edge-vs-held-signal bug from the Day 2 counter section, reappearing in a more complex context.
- **The reject alarm never triggers, even after several rejects.** Ask: “is the alarm watching the Light count, the Heavy count, or the sum of both — and which one does the spec actually ask for?” A common mistake is wiring the alarm to only one of the two reject counters instead of their combined total.
- **The diverter fires for the wrong part, one station too early or too late.** Ask: “is the diverter's trigger condition reading the classification currently sitting in the Station 3 slot of the register, or some other slot?” This is the same off-by-one register indexing issue introduced in the Day 4 checkpoint, now with real consequences on a moving part.

3.5 — Extra challenge, for pairs who finish early

- Add a PID-controlled variable-speed conveyor motor (from Day 4) that slows the line down automatically once the reject count crosses 3, giving downstream staff more time to physically clear the reject lane before it backs up.
- Package the classify-divert-count sequence as a reusable Function Block (Day 4), and instantiate it twice to represent two parallel packaging lines feeding into one shared dashboard, with independent counts and independent reject alarms per line.

End of course checkpoint

Every student/pair demonstrates the full Sorting & Packaging Line running at least 10 parts of mixed weights end-to-end without manual intervention, with correct live counts and a correctly firing reject alarm shown on the dashboard — and can explain out loud how the shift register carries a part's classification from Station 2 through to the Station 3 diverter, before the course ends.